

LDL^T Factorization of Sparse Symmetric Matrices

1st Emir Baručija

Faculty of Electrical Engineering, Sarajevo

Abstract—This project implements the sparse LDLT factorization of symmetric matrices in C++ with the up-looking algorithm from the educational LDL package of SuiteSparse. The library covers the whole pipeline: triplet and compressed-column storage, Matrix Market input and output, elimination tree with postorder and column counts, symbolic and numeric factorization, triangular solves and residual-based validation. Three fill-reducing orderings are implemented, reverse Cuthill-McKee, exact minimum degree and approximate minimum degree. Indefinite systems are handled with static pivoting followed by iterative refinement, which keeps the simple kernel and still gives solutions with backward error at machine precision. Code can be run through a command-line tool, `ldlt`, with an interactive session and batch commands, backed by an SQLite catalog of eighteen matrices from the SuiteSparse collection.

Index Terms—sparse matrices, LDLT factorization, elimination tree, minimum degree ordering, iterative refinement

I. INTRODUCTION

The LDL^T factorization writes a symmetric matrix as $A = LDL^T$, with L unit lower triangular and D diagonal. It is a variant of Cholesky that avoids square roots, so D is allowed to have negative entries and the factorization works for indefinite matrices as well. When A is sparse, the whole difficulty is in keeping L sparse: the order in which rows and columns are eliminated decides how many new nonzeros (fill-in) appear, and a good ordering can reduce the work by orders of magnitude.

The theoretical source for this project is Davis' book on direct methods [1], and the starting point for the code was the LDL package [2] from SuiteSparse. LDL alone expects a matrix that is already ordered, and offers no ordering, no protection against small pivots and no validation. In addition to that, I added COO to CSC conversion and Matrix Market I/O.

II. IMPLEMENTATION

A. Storage and input

A matrix is read from a Matrix Market file [9] into triplet (COO) form and then converted to compressed sparse column (CSC) storage, which sorts every column and sums duplicate entries. Symmetric files store only the lower triangle, so the reader expands them by default into the full matrix. This matters, because the factorization kernel reads only the upper triangle of PAP^T , and once a permutation is applied an entry that was upper in A may become lower. I ran into this early on, so a lower-only matrix, or an upper-only matrix combined with a permutation, is now rejected with an explanation instead of silently producing a wrong factor.

B. Symbolic analysis

Before any arithmetic, the pattern of L is computed from the pattern of A . The elimination tree is built with the path-compression algorithm from [1]: for every column k and every nonzero a_{ik} with $i < k$, the path from i towards the root is followed and shortcut, and the first node without a parent gets k as its parent. A depth-first postorder of the tree follows, and the column counts of L are obtained with the row-subtree skeleton algorithm of Gilbert, Ng and Peyton [5]. The column counts give the column pointers of L , the number of nonzeros and the flop count of the numeric phase, $\sum_j c_j(c_j + 2)$, before a single value of L exists. This is what makes it possible to compare orderings cheaply: the symbolic phase alone tells how much fill each ordering produces.

C. Numeric factorization and solve

The numeric phase is a port of the up-looking kernel of [2]. Row k of L is computed by a sparse triangular solve $L_{1:k-1, 1:k-1} y = a_{1:k-1, k}$, where the nonzero pattern of y is found first by walking the elimination tree from every nonzero of column k up to an already visited node. The values are then computed in that topological order, each $l_{ki} = y_i/d_i$ is stored into column i of L , and d_k is updated by $-l_{ki}y_i$. The solve phase permutes the right-hand side, performs the forward solve with L , divides by D , performs the backward solve with L^T , and permutes back.

I built two checks. The residual-based one solves $Ax = b$ for a b built from a known x_{true} and reports the forward error $\|x - x_{\text{true}}\|_{\infty}/\|x_{\text{true}}\|_{\infty}$ and the normwise backward error

$$\eta = \frac{\|b - Ax\|_{\infty}}{\|A\|_{\infty}\|x\|_{\infty} + \|b\|_{\infty}}, \quad (1)$$

as defined in [8]: a solution is as good as it can be when η is around the machine epsilon. The second check forms $PAP^T - LDL^T$ explicitly and reports its Frobenius norm relative to $\|A\|_F$. It only works for small matrices, but it tests the factor itself and not only the solve.

D. Fill-reducing orderings

All orderings work on the adjacency graph of $A + A^T$ without the diagonal. Reverse Cuthill-McKee [4] is a breadth-first search, started from a pseudo-peripheral node found with the George-Liu procedure [6], with neighbours visited in order of increasing degree and the result reversed. It reduces bandwidth rather than fill, and I keep it as a baseline. Exact minimum degree eliminates at each step a node of smallest degree in the elimination graph and connects its neighbours into a clique. I implemented it with a lazy heap and sorted

adjacency lists; it is meant as a reference, since the explicit clique formation makes it too slow for large matrices.

The one to actually use is approximate minimum degree (AMD) [3]. Instead of forming cliques, an eliminated node becomes an element, and every variable keeps a list of adjacent variables and a list of adjacent elements. When pivot p is eliminated, the elements adjacent to p are absorbed into it, so the graph never grows. The degree of a variable is not computed exactly but bounded by $|A_i| + |L_p \setminus i| + \sum_e |L_e \setminus L_p|$, where the last term comes from a single pass over the elements adjacent to L_p . Variables with identical adjacency lists are detected by hashing and merged into supervariables, which are eliminated together. Dense-row handling from the original AMD I did not implement.

E. Indefinite matrices

For an indefinite matrix a pivot d_k can be arbitrarily close to zero even when A is well conditioned, and dividing by it destroys the factor. Bunch-Kaufman pivoting [7], which chooses between a 1×1 and a 2×2 pivot is used to avoid this problem. A 2×2 pivot block, however, needs to look at two columns at once, which requires a left-looking kernel; the up-looking kernel computes one row of L at a time and has no place for it. Therefore, I went with static pivoting instead, which is what production solvers such as PARDISO do [10]. Every pivot with $|d_k| < r \|A\|_\infty$ is replaced by $\pm r \|A\|_\infty$, sign kept, so the factorization always completes. It is then the exact factorization of a slightly perturbed matrix $A + E$, and iterative refinement against the original A brings the backward error (1) back to machine precision. Refinement stops when the target is reached, when the improvement stagnates, or when the error grows, in which case the previous iterate is restored. The tool uses $r = 10^{-12}$. I first tried the classic 10^{-8} , but it perturbed pivots of badly scaled but positive definite stiffness matrices and made their results worse, while 10^{-12} was safe on every matrix in the test set.

III. COMMAND LINE TOOL

I decided to make command-line interface as application that demonstrates how the code can be used. It opens an interactive session with a built-in catalog of matrices. The catalog is an SQLite [11] database that fills itself on first run from the eighteen matrices, smallest first, so that `mtx_1` is always quick. The test set comes from the SuiteSparse Matrix Collection [12]: twelve positive definite stiffness matrices of the Harwell-Boeing group and six indefinite systems of the GHS_indef group. Typing a matrix name factorizes it and prints the dimension, the nonzeros of A and L , the fill ratio, the flop count, the time of every phase, the inertia read off the signs of D , the number of perturbed pivots, and the backward and forward error before and after refinement. Other commands switch the ordering, import a file or a whole folder, and save the full L , D and P of the last run to a text file.

REFERENCES

[1] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: SIAM, 2006.

[2] T. A. Davis, "Algorithm 849: A concise sparse Cholesky factorization package," *ACM Trans. Math. Softw.*, vol. 31, no. 4, pp. 587–591, Dec. 2005.

[3] P. R. Amestoy, T. A. Davis, and I. S. Duff, "An approximate minimum degree ordering algorithm," *SIAM J. Matrix Anal. Appl.*, vol. 17, no. 4, pp. 886–905, 1996.

[4] E. Cuthill and J. McKee, "Reducing the bandwidth of sparse symmetric matrices," in *Proc. 24th Nat. Conf. ACM*, 1969, pp. 157–172.

[5] J. R. Gilbert, E. G. Ng, and B. W. Peyton, "An efficient algorithm to compute row and column counts for sparse Cholesky factorization," *SIAM J. Matrix Anal. Appl.*, vol. 15, no. 4, pp. 1075–1091, 1994.

[6] A. George and J. W. H. Liu, *Computer Solution of Large Sparse Positive Definite Systems*. Englewood Cliffs, NJ: Prentice-Hall, 1981.

[7] J. R. Bunch and L. Kaufman, "Some stable methods for calculating inertia and solving symmetric linear systems," *Math. Comp.*, vol. 31, no. 137, pp. 163–179, 1977.

[8] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.

[9] R. F. Boisvert, R. Pozo, and K. A. Remington, "The Matrix Market exchange formats: Initial design," NIST, Gaithersburg, MD, Tech. Rep. NISTIR 5935, Dec. 1996.

[10] O. Schenk and K. Gärtner, "On fast factorization pivoting methods for sparse symmetric indefinite systems," *Electron. Trans. Numer. Anal.*, vol. 23, pp. 158–179, 2006.

[11] D. R. Hipp et al., "SQLite," version 3.53, 2026. [Online]. Available: <https://sqlite.org>

[12] T. A. Davis and Y. Hu, "The University of Florida Sparse Matrix Collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, Art. 1, Dec. 2011.